6.2 · Limpieza y preprocesado de datos

Antes de entrenar cualquier modelo de machine learning, los datos deben estar en condiciones óptimas. Esta sección cubre las técnicas esenciales para detectar y corregir defectos, transformar variables y preparar correctamente las particiones de entrenamiento y prueba.

CIENCIA DE DATOS PREPROCESADO



Defectos típicos de un dataset

En la práctica, los datos rara vez llegan limpios. Reconocer los problemas más frecuentes es el primer paso para un preprocesado eficaz.

Inconsistencias

Valores contradictorios entre columnas relacionadas o registros que violan restricciones de dominio.

Ruido

Errores de medición o introducción manual que distorsionan los patrones reales del fenómeno estudiado.

Sesgo de muestreo

La muestra no representa fielmente la población objetivo, lo que introduce errores sistemáticos en el modelo.

Datos irrelevantes

Variables que no aportan información predictiva y pueden añadir ruido o incrementar el coste computacional.



Valores nulos

Los valores nulos (`NaN`, `None`, celdas vacías) son uno de los problemas más habituales. Su tratamiento incorrecto puede sesgar el modelo o provocar errores en el entrenamiento.

Estrategias de imputación

- **Eliminación:** borrar filas o columnas si el porcentaje de nulos es elevado (>40–50%).
- **Imputación estadística:** sustituir por la media, mediana o moda según la distribución de la variable.
- **Imputación por modelo:** usar KNN o regresión para predecir el valor faltante.
- **Indicador binario:** crear una columna auxiliar que señale si el valor era nulo, preservando esa información.

Regla práctica

Antes de imputar, analiza el mecanismo de ausencia: ¿es aleatoria (*MCAR*), condicionada (*MAR*) o sistemática (*MNAR*)? El mecanismo determina la estrategia óptima.

Duplicados y tipos incorrectos

Dos problemas silenciosos que afectan directamente a la calidad del aprendizaje si no se detectan a tiempo.

Registros duplicados

Aparecen por errores de integración de fuentes o fallos en la recogida de datos. Los duplicados **inflan artificialmente** la frecuencia de ciertos patrones y sesgan las métricas de evaluación. Se detectan con `df.duplicated()` en pandas y se eliminan con `drop_duplicates()`.

Tipos de datos incorrectos

Una columna numérica almacenada como texto (`object`) impide operaciones matemáticas y transformaciones. Las fechas sin parsear bloquean la extracción de características temporales. Es imprescindible verificar con `df.dtypes` y corregir con `astype()` o `pd.to_datetime()`.

Errores tipográficos en categorías

Variantes como `"Madrid"`, `"madrid"` y `"MADRID"` se tratan como tres categorías distintas. La normalización de texto (minúsculas, strip, mapeos) evita este tipo de fragmentación artificial.



Outliers (valores atípicos)

Los *outliers* son observaciones que se alejan significativamente del comportamiento general del conjunto de datos. Pueden ser errores o señales reales — el contexto determina la acción.

1

Detección

Regla IQR, puntuación Z (*z-score*), gráficos de caja (*boxplot*) o algoritmos como Isolation Forest.

2

Decisión

¿Es un error de medición o un caso legítimo extremo? El análisis de negocio es fundamental para decidir.

3

Tratamiento

Eliminación, winsorización (recorte al percentil 5–95) o transformación logarítmica para reducir su influencia.

Escalado de características

Los algoritmos basados en distancias (KNN, SVM, redes neuronales) son sensibles a la escala de las variables. Una variable en miles de euros dominaría sobre otra en porcentajes si no se normalizan.

StandardScaler

Media 0, desviación estándar 1. Ideal para distribuciones aproximadamente normales.

MinMaxScaler

Rango [0, 1]. Útil cuando se conocen los límites de la variable.

RobustScaler

Basado en mediana e IQR. Resistente a outliers extremos.

⚠ Escalar siempre **después** de dividir en train/test. Ajustar el scaler solo con los datos de entrenamiento (`fit_transform`) y aplicarlo al test (`transform`) para evitar fuga de información.

Los modelos basados en árboles (Random Forest, XGBoost) son **invariantes al escalado** porque operan con umbrales de corte, no con distancias.

Codificación de variables categóricas

Los modelos de ML requieren entradas numéricas. Las variables categóricas deben convertirse sin introducir ordinales artificiales ni expandir excesivamente el espacio de características.



Label Encoding

Asigna un entero a cada categoría. Apto para variables **ordinales** (bajo, medio, alto). En nominales introduce un orden falso.



One-Hot Encoding

Crea una columna binaria por categoría. Ideal para nominales. Puede generar muchas columnas con alta cardinalidad (*curse of dimensionality*).



Target Encoding

Reemplaza la categoría por la media del target en ese grupo. Compacto y potente, pero requiere validación cuidadosa para evitar *leakage*.



Hashing

Aplica una función hash para reducir categorías a un espacio fijo. Útil con cardinalidades muy altas, aunque introduce colisiones.



Partición train/test

Dividir correctamente los datos es fundamental para evaluar la generalización real del modelo a datos no vistos.

Conjunto de entrenamiento (train)

Generalmente el **70–80%** de los datos. El modelo aprende los patrones a partir de este conjunto. Nunca debe contaminarse con información del test.

Conjunto de validación

Subconjunto del train (o mediante *k-fold cross-validation*) usado para ajustar hiperparámetros y seleccionar el mejor modelo durante el desarrollo.

Conjunto de prueba (test)

El **20–30%** restante. Se usa **una única vez** al final para estimar el rendimiento real. Usarlo varias veces filtra información y sesga la evaluación.

En series temporales, la partición debe ser **cronológica**: los datos más recientes van al test. Una división aleatoria introduciría *look-ahead bias*.

Fuga de datos (*Data Leakage*)

La fuga de datos es uno de los errores más graves y difíciles de detectar en ML. Ocurre cuando información del futuro o del conjunto de test se filtra durante el entrenamiento, generando modelos con métricas artificialmente optimistas que fallan en producción.

1

Leakage de preprocesado

Calcular la media para imputación o ajustar el scaler usando todo el dataset (incluido el test) antes de dividir.

2

Leakage de características

Incluir variables que son consecuencia del target o que solo se conocen después del evento que se quiere predecir.

3

Leakage temporal

En datos cronológicos, usar observaciones futuras para predecir eventos pasados al dividir aleatoriamente.

4

Leakage por grupos

Fragmentos del mismo individuo o entidad aparecen tanto en train como en test, inflando artificialmente el rendimiento.

Cómo prevenir la fuga de datos

La solución central es encapsular todo el preprocesado dentro de un **pipeline** que garantice que ninguna transformación se ajusta con datos del test.

1 Divide primero, transforma después

La división train/test debe ser el primer paso, antes de cualquier imputación, escalado o encoding.

2 Usa `Pipeline` de scikit-learn

Encadena transformadores y el estimador final. El pipeline aplica `fit_transform` solo al train y `transform` al test automáticamente.

3 Revisa la semántica de las variables

Antes de incluir una característica, verifica que su valor estaría disponible en el momento de la predicción real.

Patrón recomendado

Un pipeline típico en scikit-learn encadena: imputación → escalado → encoding → modelo. Todo el preprocesado se aprende exclusivamente de los datos de entrenamiento y se aplica de forma consistente al test, eliminando cualquier posibilidad de contaminación.

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

S06_02_limpieza — 55 min

Limpieza: duplicados, tipos, fechas, categorías, imposibles y nulos

sprints/S06/notebooks/S06_02_limpieza.ipynb